Lección 09: Dominando Modelos de IA - Técnicas Avanzadas 🧠✨

¡Bienvenido al siguiente nivel! En lecciones anteriores, aprendiste la mecánica para conectarte a una API de IA. Ahora, vamos a profundizar en cómo usar estos modelos de manera experta. Aprenderás a trabajar con imágenes, a controlar finamente el comportamiento de la IA y a construir conversaciones coherentes, transformando tus programas de simples "llamadores de API" a verdaderas aplicaciones inteligentes.

Parte 1: Más Allá del Texto - Modelos Multimodales

Los modelos de IA modernos no solo trabajan con texto. Pueden "ver" imágenes y "crear" imágenes. A esto se le llama multimodalidad.

01: Generación de Imágenes (Texto a Imagen con DALL-E)

Podemos pedirle a un modelo como DALL-E que cree una imagen a partir de una descripción de texto. La interacción es similar, pero el endpoint y los parámetros cambian, y la respuesta que recibimos es una URL a la imagen generada.

El Proceso:

1. Se usa el endpoint de generations de imágenes.
2. Enviamos un prompt describiendo la imagen deseada.
3. Especificamos el model, el size (tamaño) y otros parámetros.
4. La API nos devuelve un objeto JSON que contiene la URL de la imagen creada.

<!-- end list -->

```
import requests
import os

API_KEY = os.getenv("OPENAI_API_KEY") # Es una buena práctica usar
variables de entorno
API_URL = "https://api.openai.com/v1/images/generations"

headers = {
    "Authorization": f"Bearer {API_KEY}"
}

data = {
    "model": "dall-e-3",
    "prompt": "Un astronauta con un traje cromado de estilo retro,
montando un perezoso gigante en un desierto de Marte con dos lunas en
el cielo.",
    "n": 1, # Número de imágenes a generar
    "size": "1024x1024",
    "quality": "standard"
```

```
}

try:
    response = requests.post(API_URL, headers=headers, json=data)
    response.raise_for_status()

    result = response.json()
    image_url = result['data'][0]['url']

    print("¡Imagen generada con éxito!")
    print(f"URL de la imagen: {image_url}")

except requests.exceptions.RequestException as e:
    print(f"Error al generar la imagen: {e}")
```

02: Visión (Imagen a Texto con GPT-4o)

También podemos hacer lo contrario: darle una imagen al modelo y hacerle preguntas sobre ella. Para esto, modificamos la estructura del mensaje para incluir la URL de la imagen.

El Proceso:

1. Usamos el endpoint de chat/completions, como antes.
2. En la lista messages, el contenido del usuario ya no es solo texto, sino una lista que contiene un bloque de texto y un bloque de imagen.
3. La IA analizará la imagen en el contexto del prompt de texto.

<!-- end list -->

```
import requests
import os

API_KEY = os.getenv("OPENAI_API_KEY")
API_URL = "https://api.openai.com/v1/chat/completions"

headers = {
    "Authorization": f"Bearer {API_KEY}",
    "Content-Type": "application/json"
}

data = {
    "model": "gpt-4o",
    "messages": [
        {
            "role": "user",
            "content": [
                {
                    "type": "text",
                    "text": "¿Qué animal es este y qué está haciendo?
Describe la escena."
```

```
                },
                {
                    "type": "image_url",
                    "image_url": {
                        "url":
"https://upload.wikimedia.org/wikipedia/commons/thumb/4/4d/Cat_Novembe
r_2010-1a.jpg/1200px-Cat_November_2010-1a.jpg"
                    }
                }
            ]
        }
    ]
}

try:
    response = requests.post(API_URL, headers=headers, json=data)
    response.raise_for_status()

    result = response.json()
    description = result['choices'][0]['message']['content']

    print("Análisis de la imagen:")
    print(description)

except requests.exceptions.RequestException as e:
    print(f"Error al analizar la imagen: {e}")
```

Parte 2: El Arte del Control - Parámetros Avanzados

Un prompt es solo el principio. Podemos afinar la respuesta de la IA usando parámetros adicionales en nuestras peticiones.

temperature: El Termostato de la Creatividad

Este parámetro, que va de 0.0 a 2.0, controla qué tan predecible o creativa es la respuesta.

- **Bajo (~0.2):** Más determinista y enfocado. Ideal para tareas basadas en hechos, como resúmenes o extracción de datos.
- **Alto (~1.0):** Más aleatorio y creativo. Ideal para lluvia de ideas, escritura creativa o generar variedad.

<!-- end list -->

```
# (Usando la clase AI_API de la lección 07 para simplificar)
# ... (definición de la clase AI_API aquí) ...

# Suponiendo que modificamos el método .call() para aceptar **kwargs
# def call(self, prompt, **kwargs):
#   data = { ... }
```

```
#   data.update(kwargs) # Añade los parámetros extra
#   ...

# Llamada con baja temperatura (respuesta más probable y directa)
respuesta_directa = openai_client.call(
    "¿Cuál es la capital de Francia?",
    temperature=0.1
)
print(f"Respuesta Directa (temp=0.1): {respuesta_directa}")

# Llamada con alta temperatura (respuesta más creativa, podría
desviarse)
respuesta_creativa = openai_client.call(
    "Escribe una frase sobre la capital de Francia.",
    temperature=0.9
)
print(f"Respuesta Creativa (temp=0.9): {respuesta_creativa}")
```

max_tokens: Control de Longitud y Costos

Este parámetro establece el número máximo de "tokens" (piezas de palabras) que el modelo puede generar en su respuesta. Es crucial para:

1. **Controlar costos:** La mayoría de las APIs cobran por token.
2. **Evitar respuestas demasiado largas:** Asegura que la salida sea concisa.

<!-- end list -->

```
# (Usando una versión modificada de AI_API.call)

respuesta_corta = openai_client.call(
    "Resume la trama de 'El Señor de los Anillos' en una sola frase.",
    max_tokens=30 # Limitamos la respuesta a unos 30 tokens
)
print(f"Resumen Corto (max_tokens=30): {respuesta_corta}")
```

Parte 3: Prompt Engineering y Manejo de Conversación

El Rol del "System Prompt"

Además del rol "user", existe el rol "system". Es el primer mensaje que se envía y sirve para establecer la personalidad, las reglas y el contexto de la IA para toda la conversación. Es la herramienta más poderosa del Prompt Engineering.

```
# (Usando la clase AI_API de la lección 07)
# ... (suponiendo que el método .call ahora acepta una lista de
mensajes) ...
```

```
# Sin system prompt
mensajes_sin_contexto = [
    {"role": "user", "content": "Traduce 'I love programming' al
español."}
]
print(f"Sin System Prompt:
{openai_client.call(mensajes_sin_contexto)}")

# Con system prompt
mensajes_con_contexto = [
    {"role": "system", "content": "Eres un traductor que solo habla
como un pirata del siglo XVII."},
    {"role": "user", "content": "Traduce 'I love programming' al
español."}
]
print(f"Con System Prompt:
{openai_client.call(mensajes_con_contexto)}")
```

Construyendo un Chatbot: Manejo del Historial

Para que un chatbot "recuerde" lo que se ha dicho, debemos enviarle el historial completo de la conversación en cada nueva petición.

La Lógica:

1. Mantén una lista que almacene todos los mensajes (system, user, assistant).
2. Cuando el usuario envíe un nuevo mensaje, añádelo a la lista.
3. Envía la lista completa a la API.
4. Cuando la API responda, añade también su respuesta (con el rol "assistant") a la lista.
5. Repite.

<!-- end list -->

```
# (Usando la clase AI_API)
historial_mensajes = [
    {"role": "system", "content": "Eres un asistente servicial y
conciso."}
]

while True:
    entrada_usuario = input("Tú: ")
    if entrada_usuario.lower() == "salir":
        break

    # 1. Añade el nuevo mensaje del usuario al historial
    historial_mensajes.append({"role": "user", "content":
entrada_usuario})

    # 2. Llama a la API con el historial completo
```

```
    respuesta_ia = openai_client.call(historial_mensajes)
    print(f"IA: {respuesta_ia}")

    # 3. Añade la respuesta de la IA al historial para la próxima
vuelta
    historial_mensajes.append({"role": "assistant", "content":
respuesta_ia})
```

Parte 4: Simplificando la Interacción con SDKs

Hacer peticiones manuales con requests está muy bien para aprender, pero para proyectos más serios, es mejor usar el **SDK (Kit de Desarrollo de Software)** oficial del proveedor. Es una librería de Python que simplifica todo este proceso.

Ejemplo con la librería openai

Primero, la instalamos: pip install openai. El código para hacer una llamada se vuelve mucho más limpio y legible.

```
from openai import OpenAI
import os

# El cliente se configura una vez y se reutiliza.
# Automáticamente busca la API Key en la variable de entorno
OPENAI_API_KEY.
client = OpenAI()

# El historial se maneja de la misma forma
historial_mensajes = [
    {"role": "system", "content": "Eres un asistente que solo responde
con haikus."},
    {"role": "user", "content": "Háblame sobre Python."}
]

# La llamada a la API es mucho más directa
response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=historial_mensajes,
    temperature=0.7
)

# El acceso a la respuesta también es más limpio
print(response.choices[0].message.content)
```